

When Poison Fails After Retrieval: Revisiting Corpus Poisoning under Chunking and Reranking Pipelines

Xi Nie¹, Hongwei Li¹, Shenghao Wu¹, Mingxuan Li², Jiachen Li³, Wenbo Jiang^{1*}

¹ School of Computer Science and Engineering, University of Electronic Science and Technology of China, China

² School of Criminal Investigation, People's Public Security University of China, China

³ School of Computer Science and Artificial Intelligence, Wuhan University of Technology, China

Abstract—Retrieval-Augmented Generation (RAG) systems are vulnerable to corpus poisoning attacks that manipulate downstream model outputs through malicious knowledge injection. Existing studies mainly evaluate poisoning under simplified retrieval settings, overlooking practical RAG pipelines involving document chunking, dense retrieval, reranking, and grounded generation. In this paper, we revisit corpus poisoning under realistic multi-stage retrieval pipelines and show that many existing attacks substantially degrade after reranking despite achieving high retrieval-stage relevance. We identify retrieval granularity mismatch as a key reason for this failure: document-level adversarial signals are often fragmented during chunking, while rerankers favor locally coherent and answer-bearing passages rather than globally optimized semantic similarity. Based on this observation, we propose Chunk-aware and Rerank-Consistent Poisoning (CRCP), a poisoning framework that jointly optimizes retrieval relevance, reranker consistency, and chunk-boundary robustness. CRCP explicitly models chunking transformations during optimization to generate locally self-contained adversarial passages that remain effective under varying chunking configurations. Experiments on standard RAG benchmarks with multiple retrievers and rerankers show that existing poisoning methods are highly sensitive to chunk size and reranking strategies, whereas CRCP achieves substantially higher attack success rates and stronger robustness across realistic retrieval pipelines. Our findings highlight an important realism gap in current RAG security evaluation and suggest that poisoning in modern RAG systems should be studied as a multi-stage retrieval consistency problem rather than a retrieval-only problem.

Index Terms—RAG, Data Security, Trustworthy AI, Corpus Poisoning, Retrieval Robustness

I. INTRODUCTION

Retrieval-Augmented Generation (RAG) [1]–[3] has become a widely adopted paradigm for enhancing large language models (LLMs) with external and updatable knowledge sources. By retrieving relevant documents from external corpora during inference, RAG systems can improve factuality, reduce hallucination, and support knowledge-intensive tasks. Modern RAG systems are increasingly deployed in practical applications ranging from customer support systems to domain-specific copilots [4]–[6].

Despite these advantages, recent studies have shown that RAG systems are vulnerable to corpus poisoning attacks, where adversaries inject malicious or manipulated documents into the retrieval corpus to influence downstream model outputs [7]–[11]. Existing poisoning methods primarily optimize

adversarial documents to maximize retrieval relevance with respect to target queries, enabling poisoned content to be retrieved during inference.

However, most existing studies evaluate poisoning attacks under highly simplified retrieval settings that differ substantially from practical RAG deployments. In particular, prior methods commonly assume document-level indexing and single-stage dense retrieval, while modern production RAG systems typically employ multi-stage retrieval pipelines consisting of document chunking, dense retrieval, and cross-encoder reranking. In these systems, retrieval operates on chunk-level representations rather than full documents, and rerankers further filter retrieved candidates based on fine-grained relevance estimation. As a result, achieving high retrieval similarity at the document level does not necessarily guarantee that poisoned content survives the entire retrieval pipeline.

In this paper, we revisit corpus poisoning under realistic RAG retrieval pipelines and identify an important but underexplored issue: retrieval granularity mismatch. Existing poisoning methods optimize adversarial signals at the document level, yet practical systems retrieve and rerank chunk-level passages. Consequently, adversarial signals are frequently fragmented or diluted during chunking, especially under varying chunk sizes. Moreover, rerankers favor locally coherent and answer-bearing passages rather than globally optimized semantic similarity, introducing an additional mismatch between dense retrieval objectives and final context selection.

Through systematic empirical analysis, we show that many existing poisoning attacks degrade substantially after reranking despite achieving high retrieval-stage relevance. We further demonstrate that poisoning effectiveness is highly sensitive to chunk size and reranking strategy, leading to unstable attack performance across realistic deployment settings. These findings suggest that poisoning in modern RAG systems should no longer be viewed solely as a retrieval problem, but rather as a multi-stage retrieval consistency problem shaped by retrieval granularity and reranking dynamics.

To address this issue, we propose Chunk-aware and Rerank-Consistent Poisoning (CRCP), a realistic poisoning framework designed for modern multi-stage RAG pipelines. CRCP jointly optimizes retrieval relevance, reranker consistency,

CRCP: Chunk-aware and Rerank-Consistent Poisoning Attack Overview

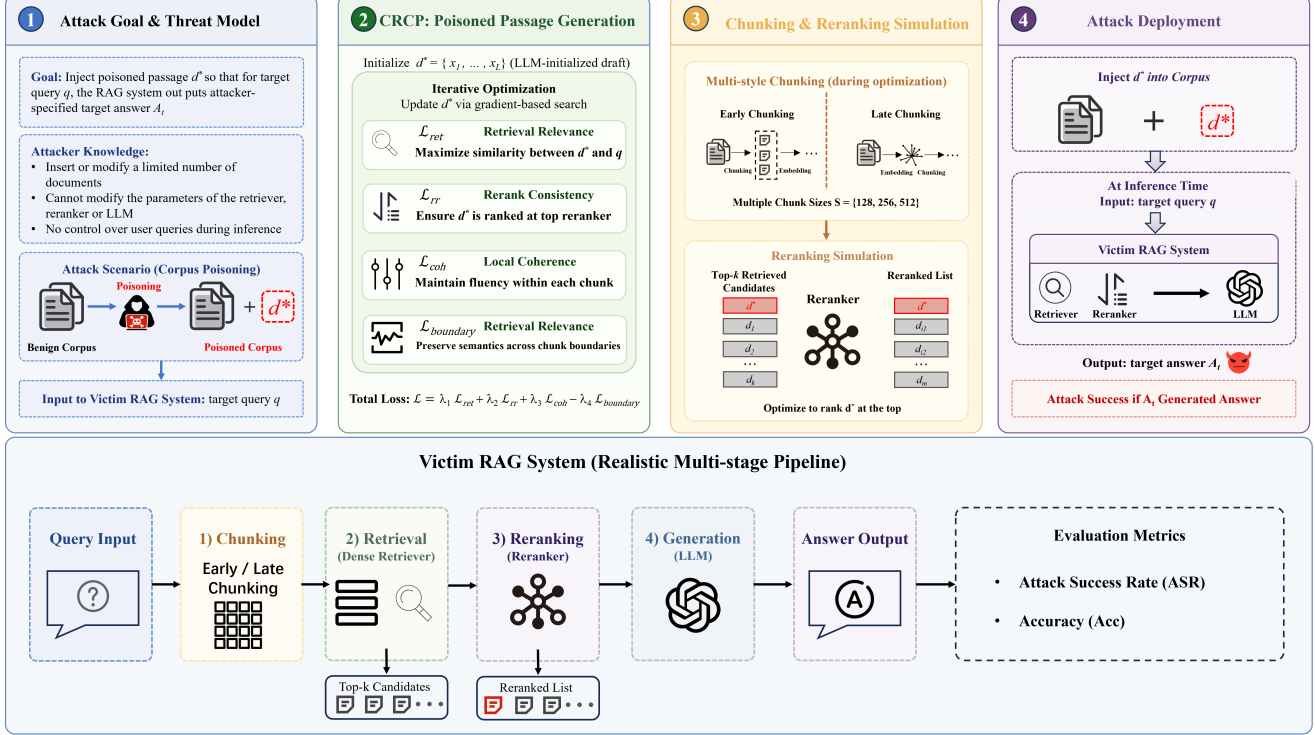


Fig. 1: An overview of the CRCP Attack.

and chunk-boundary robustness. Unlike prior methods that optimize global document-level similarity, our approach explicitly models chunking transformations during optimization and encourages locally self-contained adversarial passages that remain effective across varying chunking configurations.

Extensive experiments on standard RAG benchmarks using multiple retrievers and rerankers demonstrate that existing poisoning methods exhibit severe performance degradation under realistic retrieval settings, whereas CRCP consistently achieves higher attack success rates and stronger robustness across diverse chunking and reranking configurations. Our study highlights a significant realism gap in current RAG security evaluation and provides a more faithful framework for understanding adversarial retrieval behavior in practical RAG systems.

Our main contributions are summarized as follows:

- We revisit corpus poisoning under realistic multi-stage RAG pipelines and identify retrieval granularity mismatch as a fundamental limitation of existing poisoning methods.
- We propose CRCP, a chunk-aware and rerank-consistent poisoning framework that jointly optimizes retrieval relevance, reranker consistency, and chunk-boundary robustness.
- We demonstrate through extensive experiments that CRCP achieves substantially stronger robustness and attack effectiveness across realistic retrieval pipelines

compared with existing poisoning approaches.

II. RELATED WORK

A. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [12], [13] enhances Large Language Models (LLMs) by retrieving external knowledge to support grounded generation. Typical RAG pipelines consist of document indexing, dense retrieval, and response generation. To improve retrieval granularity and fit context-length constraints, documents are usually divided into smaller chunks before indexing. Dense retrievers then retrieve relevant chunks according to embedding similarity, while the generator produces responses conditioned on the retrieved context.

Modern RAG systems [14]–[17] commonly include reranking modules to improve retrieval precision. After the initial dense retrieval stage, rerankers, often implemented with cross-encoder architectures, reorder candidate passages based on fine-grained semantic relevance. Unlike dense retrievers that mainly optimize embedding similarity, rerankers prefer locally coherent and directly answer-bearing passages. Therefore, passages with high retrieval similarity may still be filtered out during reranking.

Recent studies suggest that chunking and reranking significantly influence downstream retrieval behavior. Since retrieval is performed on chunks rather than full documents, document-level semantic signals may be fragmented under different chunking strategies, leading to retrieval inconsistency. More-

over, rerankers introduce an additional selection mechanism that can suppress adversarial passages lacking local semantic coherence.

B. Attacks in RAG Systems

With the increasing deployment of RAG systems, security researchers have begun to investigate the vulnerabilities and attack surfaces of RAG pipelines. Existing studies have proposed various corpus poisoning strategies targeting different stages of the retrieval-generation process. For instance, AgentPoison [18] optimizes trigger-document pairs to increase the likelihood that poisoned content is retrieved. PoisonedRAG [19] injects target queries into poisoned documents to maximize retrieval probability, thereby inducing the LLM to generate misleading or incorrect responses. LIAR [20] considers a corpus-aware adversarial setting, leveraging non-target documents to improve attack robustness and stability. Joint-GcG [21] unifies the attack surface by jointly optimizing objectives across both the retriever and generator. Paradox [22] introduces a contrastive triplet generation mechanism for crafting adversarial samples. CtrlRAG [10] proposes a two-stage black-box attack framework that demonstrates strong effectiveness in hallucination amplification and emotion manipulation tasks across RAG systems. CPA-RAG [23] further investigates covert poisoning attacks; however, balancing retrieval effectiveness with textual stealthiness remains a fundamental challenge for coarse-grained poisoning strategies.

C. Existing Defense Strategies

A wide range of defense mechanisms has been proposed to mitigate adversarial threats against RAG systems. RevPRAG [24] detects poisoned inputs by analyzing abnormal activation patterns within LLMs. RobustRAG [25] improves system robustness through an isolate-then-aggregate framework that decouples retrieval paths to reduce the influence of poisoned content. AstuteRAG [26] adaptively integrates retrieved information with the model’s internal knowledge via heuristic-based selection strategies. InstructRAG [27] enhances retrieval-augmented generation by leveraging self-synthesized rationales to guide the retrieval process, thereby improving the relevance and coherence of generated responses. TrustRAG [28] mitigates malicious content through embedding-based document clustering and further introduces a consistency-driven conflict resolution mechanism. SeCon-RAG [29] adopts a two-stage framework that combines fine-grained semantic filtering with conflict-aware inference to improve robustness against adversarial retrieval content.

III. THREAT MODEL

A. Adversary Goals

The adversary aims to manipulate the final outputs of a Retrieval-Augmented Generation (RAG) system by injecting poisoned documents into the external knowledge base. Given a target query, the attacker seeks to ensure that the poisoned content is retrieved, survives reranking, and ultimately influences the generated response toward attacker-intended outputs.

Unlike prior work that mainly optimizes retrieval-stage similarity, we consider poisoning under realistic multi-stage RAG pipelines involving chunking, dense retrieval, reranking, and grounded generation. Therefore, the attack objective is to maximize end-to-end poisoning effectiveness across the entire retrieval pipeline rather than only improving retriever relevance scores.

B. Adversary Capabilities

We assume a corpus poisoning adversary who can insert or modify a limited number of documents in the external knowledge corpus. The attacker cannot modify the parameters of the retriever, reranker, or language model, and has no control over user queries during inference.

We distinguish two attacker capability levels. **White-box surrogate** (primary setting): the attacker holds open-source surrogate models of the same architecture family as the victim, enabling gradient-based optimization via backpropagation through the surrogate cross-encoder. **Score-only black-box** (fallback): the attacker can only query reranker scores, in which case CRCP approximates $\mathcal{L}_{rr}$ gradients via finite differences over top- κ token candidates. In neither case does the attacker access victim model weights or exact chunking configurations at inference time. Since practical RAG systems may employ different chunk sizes and segmentation strategies, the attacker must generate poisoned passages that remain effective under varying chunk boundaries and reranking behaviors.

IV. METHODOLOGY

A. Problem Formulation

1) *RAG Retrieval Pipeline*: We consider a practical Retrieval-Augmented Generation (RAG) system consisting of four stages: document chunking, dense retrieval, reranking, and grounded generation. Given a user query q , the system first partitions each document D in the knowledge base into a set of chunks:

$$\mathcal{C}(D) = c_1, c_2, \dots, c_n, \quad (1)$$

where the chunking operation depends on a chunking policy $\phi(\cdot)$ parameterized by chunk size and splitting strategy. The entire chunk corpus is denoted as:

$$\mathcal{C} = \bigcup_{D \in \mathcal{D}} \mathcal{C}(D). \quad (2)$$

For a query q , a dense retriever $f_r(\cdot)$ computes similarity scores between the query embedding and chunk embeddings:

$$s_r(q, c) = \text{sim}(f_r(q), f_r(c)). \quad (3)$$

The top- k retrieved chunks are then passed into a reranker $f_{rr}(\cdot)$, which produces refined relevance scores:

$$s_{rr}(q, c) = f_{rr}(q, c). \quad (4)$$

The reranker selects the final top- m chunks:

t-SNE Visualization: Embedding Distribution Comparison across Poisoning Methods and Retrieval Granularities

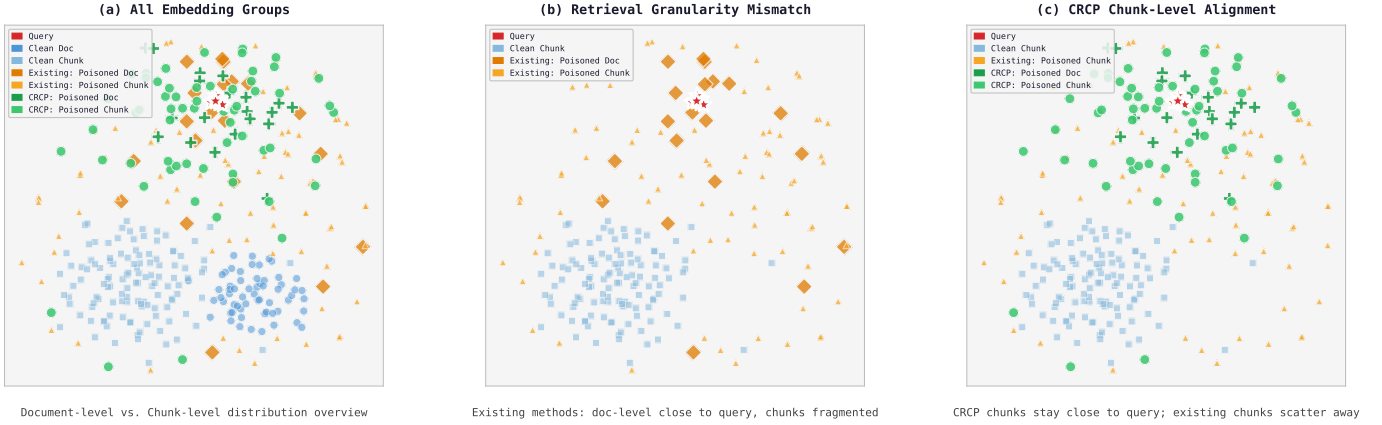


Fig. 2: t-SNE Visualization: Embedding Distribution Comparison across Poisoning Methods and Retrieval Granularities.

$$\mathcal{C}^* = \text{TopM } c \in \text{TopK}(\mathcal{C}) \text{ srr}(q, c). \quad (5)$$

Finally, the generator $f_g(\cdot)$ conditions on the selected chunks to produce the final response:

$$y = f_g(q, \mathcal{C}^*). \quad (6)$$

2) *Corpus Poisoning Attack*: In corpus poisoning attacks, the adversary injects a set of poisoned documents:

$$\mathcal{D}_{adv} = \mathcal{D}_{adv}^1, \mathcal{D}_{adv}^2, \dots, \mathcal{D}_{adv}^t \quad (7)$$

into the external corpus. The attack objective is to manipulate the generated response toward an attacker-desired target behavior when victim queries are issued.

Prior poisoning attacks mainly optimize retrieval-stage similarity, encouraging poisoned documents to achieve high dense retrieval relevance:

$$\max_{\mathcal{D}_{adv}} s_r(q, \mathcal{D}_{adv}). \quad (8)$$

However, modern RAG systems no longer directly retrieve full documents. Instead, chunking and reranking introduce additional transformations and selection constraints. Consequently, document-level adversarial optimization may not survive downstream retrieval stages.

3) *Retrieval Granularity Mismatch*: We define retrieval granularity mismatch as the inconsistency between the optimization granularity of poisoning attacks and the actual processing granularity of modern RAG pipelines.

Existing poisoning attacks typically optimize adversarial relevance at the document level. However, after chunking, the adversarial signal is distributed across multiple chunks:

$$\mathcal{D}_{adv} \xrightarrow{\phi(\cdot)} c_1, c_2, \dots, c_n. \quad (9)$$

Because rerankers evaluate local passage quality and query-answer consistency, chunks containing incomplete adversarial

semantics or weak local coherence may receive low reranking scores even if the original document achieves high retrieval similarity.

As a result, the attack pipeline suffers from two major failure modes:

- **Chunk fragmentation failure**: adversarial semantics are split across chunk boundaries, reducing local relevance. (see Fig. 2(b)).
- **Reranking inconsistency failure**: chunks optimized for dense retrieval similarity are not necessarily preferred by cross-encoder rerankers.

Empirical Quantification via Signal Retention Rate. To empirically validate the retrieval granularity mismatch hypothesis, we introduce the *Signal Retention Rate* (SRR):

$$\text{SRR}(\phi, \mathcal{D}_{adv}, q) = \frac{\max_{c \in \phi(\mathcal{D}_{adv})} s_r(q, c)}{s_r(q, \mathcal{D}_{adv})}, \quad (10)$$

which measures the fraction of document-level adversarial signal preserved in the best chunk after applying chunking policy ϕ . $\text{SRR} \approx 1$ indicates complete preservation; $\text{SRR} \ll 1$ reveals severe fragmentation.

As shown in Figure 3, existing methods suffer dramatic SRR degradation as chunk size decreases. At chunk size 128, PoisonedRAG, Joint-GCG, and RAG Paradox achieve median SRR of only 0.319, 0.297, and 0.343, respectively, with **100%** of documents falling below the empirical reranking survival threshold of 0.70 — the SRR level at which rerankers reliably promote a chunk into the top- m selection. The median SRR drop from chunk size 512 to 128 reaches 0.387–0.394 for all three baselines, reflecting extreme sensitivity to chunking granularity. In contrast, CRCP maintains median SRR of 0.807 at chunk size 128 and 0.947 at chunk size 512, with only 7% of documents below the survival threshold at the smallest chunk size and 0% at larger sizes. All pairwise comparisons between CRCP and baselines are statistically significant (Mann-Whitney U , $p < 0.001$).

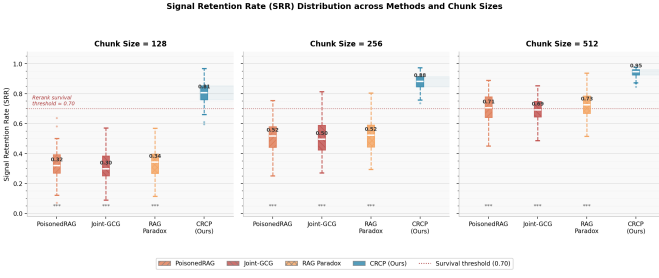


Fig. 3: Distribution of Signal Retention Rate (SRR) across poisoning methods and chunk sizes (NQ, $N=100$ documents). CRCP remains above threshold across all settings.

Our objective is therefore to design poisoning documents that remain adversarially effective after chunking and reranking transformations. Fig. 2(c) previews that CRCP achieves this by keeping chunk-level embeddings tightly aligned with target queries.

B. Chunk-aware and Rerank-Consistent Poisoning (CRCP)

To address the above limitations, we propose Chunk-aware and Rerank-Consistent Poisoning (CRCP), a poisoning framework that jointly optimizes:

- 1) Dense retrieval relevance;
- 2) Reranker consistency;
- 3) Chunk-boundary robustness.

The overall framework is illustrated as a multi-stage optimization process where poisoning effectiveness is preserved across realistic retrieval pipelines.

1) *Design Principles*: CRCP is motivated by three observations.

First, retrieval relevance alone is insufficient in realistic RAG systems because rerankers act as a second-stage semantic filter. A poisoning document must therefore remain competitive under both retriever and reranker scoring.

Second, chunking transformations are inherently unstable across practical systems. Different chunk sizes can substantially alter semantic completeness within each chunk. Robust poisoning should therefore survive diverse chunking configurations.

Third, rerankers tend to favor locally self-contained passages that explicitly answer the query. Consequently, adversarial content should be distributed into semantically coherent local units rather than relying on global document-level optimization.

Based on these observations, CRCP performs chunk-aware optimization directly over chunk-level adversarial passages.

C. Chunk-aware Poisoning Optimization

1) *Chunk Transformation Modeling*: Instead of optimizing poisoning documents under a fixed chunking configuration, CRCP explicitly models chunking as a stochastic transformation process.

Let:

$$\Phi = \phi_1, \phi_2, \dots, \phi_M \quad (11)$$

represent a set of possible chunking strategies with different chunk sizes.

For a poisoning document D_{adv} , each chunking policy generates a chunk set:

$$\mathcal{C}\phi_i(D_{adv}) = c_1^{(i)}, c_2^{(i)}, \dots \quad (12)$$

CRCP optimizes poisoning effectiveness across all chunking transformations:

$$\max_{D_{adv}} \mathbb{E} \phi \sim \Phi \left[\mathcal{L}_{attack}(q, \phi(D_{adv})) \right]. \quad (13)$$

This formulation encourages adversarial semantics to remain stable even when chunk boundaries change.

2) *Local Semantic Self-Containment*: To reduce chunk fragmentation, CRCP enforces local semantic completeness within each chunk.

Instead of distributing attack semantics globally across the document, we construct poisoning passages such that each local chunk independently contains:

- 1) query-relevant context,
- 2) answer-bearing semantics,
- 3) adversarial guidance signals.

Formally, for each chunk c_i , CRCP maximizes local query relevance:

$$\mathcal{L}_{local} = s_r(q, c_i). \quad (14)$$

Simultaneously, we encourage semantic coherence within the chunk using an intra-chunk consistency objective:

$$\mathcal{L}_{coh} = \frac{1}{|c_i|} \sum_j \text{Cos}_{sim}(E_r(w_j^{\text{context}}), E_r(q)) \quad (15)$$

where $\text{Coh}(\cdot)$ measures local semantic consistency using contextual embedding similarity.

The resulting poisoning chunks become locally self-contained adversarial passages that are less sensitive to chunk-boundary changes.

3) *Boundary Robustness Optimization*: Chunk boundaries may arbitrarily truncate important adversarial signals. To mitigate this issue, CRCP introduces boundary robustness optimization.

Specifically, during optimization, we randomly shift chunk boundaries and evaluate whether adversarial semantics remain preserved:

$$\tilde{c}_i = \text{ShiftBoundary}(c_i, \delta), \quad (16)$$

where δ is a random offset.

The optimization objective encourages consistent retrieval relevance under shifted boundaries:

$$\mathcal{L}_{boundary} = \mathbb{E} \delta \left[|s_r(q, c_i) - s_r(q, \tilde{c}_i)| \right]. \quad (17)$$

Minimizing this loss reduces sensitivity to chunk partition perturbations and improves transferability across practical chunking strategies.

D. Rerank-Consistent Optimization

1) *Reranker-aware Adversarial Objective:* Dense retrievers and rerankers exhibit substantially different relevance preferences.

Dense retrievers mainly rely on embedding-level semantic similarity, while rerankers evaluate fine-grained token-level query-passage interactions. Consequently, adversarial passages optimized solely for dense retrieval may appear semantically unnatural or weakly answer-bearing to rerankers.

To address this inconsistency, CRCP jointly optimizes reranker relevance.

For each adversarial chunk c_i , the reranker consistency objective is defined as:

$$\mathcal{L}_{rr} = srr(q, c_i). \quad (18)$$

This objective encourages poisoning chunks to remain competitive during reranking.

2) *Joint Multi-stage Optimization:* CRCP jointly optimizes dense retrieval relevance, reranker consistency, local coherence, and boundary robustness.

The overall optimization objective is:

$$\mathcal{L}_{CRCP} = \lambda_1 \mathcal{L}_{ret} + \lambda_2 \mathcal{L}_{rr} + \lambda_3 \mathcal{L}_{coh} - \lambda_4 \mathcal{L}_{boundary}, \quad (19)$$

where:

$$\mathcal{L}_{ret} = \mathbb{E} \phi \sim \Phi[s_r(q, \phi(D_{adv}))]. \quad (20)$$

The coefficients $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ control the trade-off between retrieval relevance, reranking effectiveness, semantic coherence, and chunk robustness.

This joint objective explicitly treats corpus poisoning as a multi-stage retrieval consistency problem rather than a retrieval-only optimization problem.

E. Poisoning Document Construction

1) *Query-conditioned Passage Generation:* Given a target query set:

$$\mathcal{Q} = q_1, q_2, \dots, q_n, \quad (21)$$

CRCP first constructs query-conditioned poisoning passages.

For each target query, we generate passages that:

- 1) explicitly mention query-relevant concepts,
- 2) contain locally complete answer patterns,
- 3) embed attacker-desired semantic guidance.

Unlike prior approaches that rely heavily on keyword stuffing or global semantic similarity optimization, CRCP emphasizes natural local answerability to better align with reranker preferences.

Algorithm 1 CRCP Poisoning

Input: Target query set $\mathcal{Q} = \{q_1, q_2, \dots, q_n\}$, target response a_{target} ,
 1: chunking strategy set $\Phi = \{\phi_1, \phi_2, \dots, \phi_M\}$,
 2: dense retriever $f_r(\cdot)$, cross-encoder reranker $f_{rr}(\cdot)$,
 3: loss weights $\lambda_1, \lambda_2, \lambda_3, \lambda_4 > 0$, boundary offset bound δ_{max} ,
 4: optimization iterations T , top- κ candidate tokens per step.
Output: Adversarial poisoning document D_{adv} .

Phase 1: Query-conditioned Initialization

5: Generate draft $D_{\text{adv}}^{(0)}$ via prompted LLM with $(\mathcal{Q}, a_{\text{target}})$

Phase 2: Joint Multi-stage Optimization

```

6: for  $t = 1$  to  $T$  do
7:    $\mathcal{L} \leftarrow 0$ 
8:   for each  $q_i \in \mathcal{Q}$  do
9:     for each  $\phi_j \in \Phi$  do
10:       $\{c_1^{(j)}, c_2^{(j)}, \dots\} \leftarrow \phi_j(D_{\text{adv}}^{(t-1)})$ 
11:      for each chunk  $c_k^{(j)}$  do
12:         $\mathcal{L}_{ret} \leftarrow s_r(q_i, c_k^{(j)})$ 
13:         $\mathcal{L}_{rr} \leftarrow s_{rr}(q_i, c_k^{(j)})$ 
14:         $\mathcal{L}_{coh} \leftarrow \frac{1}{|c_k^{(j)}|} \sum_p \cos(\mathbf{e}_p^{\text{ctx}}, f_r(q_i))$ 
15:        Sample  $\delta \sim \mathcal{U}[-\delta_{\text{max}}, \delta_{\text{max}}]$ 
16:         $\tilde{c}_k^{(j)} \leftarrow \text{SHIFTBOUNDARY}(c_k^{(j)}, \delta)$ 
17:         $\mathcal{L}_{boundary} \leftarrow |s_r(q_i, c_k^{(j)}) - s_r(q_i, \tilde{c}_k^{(j)})|$ 
18:         $\mathcal{L} += \lambda_1 \mathcal{L}_{ret} + \lambda_2 \mathcal{L}_{rr} + \lambda_3 \mathcal{L}_{coh} - \lambda_4 \mathcal{L}_{boundary}$ 
19:      end for
20:    end for
21:  end for
22:   $\mathbf{g} \leftarrow \nabla_{D_{\text{adv}}} \mathcal{L}$ 
23:   $D_{\text{adv}}^{(t)} \leftarrow \text{TOPK-TOKENSUBSTITUTE}(D_{\text{adv}}^{(t-1)}, \mathbf{g}, \kappa)$ 
24: end for
```

Phase 3: Post-processing

```

25:  $D_{\text{adv}} \leftarrow \text{DOCUMENTCOMPOSITION}(D_{\text{adv}}^{(T)})$ 
26: return  $D_{\text{adv}}$ 
```

Concretely, we initialize each poisoning passage using a prompted LLM (GPT-4o in our experiments) with the following structured template:

```

[TARGET QUERY: {q}]
[TARGET ANSWER: {a_target}]
Generate a factual-sounding passage of
approximately  $L$  words that naturally
discusses {q} and leads to the
conclusion {a_target}.
The passage should be locally
self-contained, explicitly
answer-bearing, and written in a
neutral encyclopedic style consistent
with knowledge base documents.
```

This LLM-generated draft serves as the initialization for subsequent gradient-based optimization (Algorithm 1, Phase 1). A semantically coherent initialization is critical: it provides a warm start that satisfies reranker preferences for local answerability from the outset, helping the optimizer avoid degenerate solutions that achieve high retrieval similarity via incoherent token sequences yet are subsequently filtered by

TABLE I: Acc and ASR in RAG Systems with Retriever-Only Pipelines.

Dataset	Method	Accuracy ($\downarrow$ better)						ASR ($\uparrow$ better)					
		GPT-4o	DeepSeek-R1	Qwen3-Max	Qwen2.5-7B	Vicuna-7B	LLaMA-2-7B	GPT-4o	DeepSeek-R1	Qwen3-Max	Qwen2.5-7B	Vicuna-7B	LLaMA-2-7B
NQ	Clean	46.15	46.21	37.92	35.81	37.39	—	—	—	—	—	—	—
	PoisonedRAG-BB(USENIX 25)	32.60	34.91	34.32	30.09	31.22	33.18	88.20	86.76	85.67	93.12	93.26	91.31
	The RAG Paradox(EMNLP 25)	18.37	18.10	20.02	20.17	19.05	14.32	90.17	93.65	91.77	93.25	93.10	94.05
	Joint-GCG(AAAI 26)	20.01	22.45	22.47	19.09	17.34	19.02	94.33	93.19	91.68	96.01	95.65	90.37
	Ours	15.41	15.63	18.12	18.99	16.36	14.21	92.81	91.27	95.74	92.89	95.01	93.28
HotpotQA	Clean	49.92	50.06	53.29	51.05	47.39	48.16	—	—	—	—	—	—
	PoisonedRAG-BB(USENIX 25)	27.26	25.13	26.19	21.37	21.22	20.08	87.56	88.73	88.50	89.36	88.71	86.18
	The RAG Paradox(EMNLP 25)	19.01	19.78	18.92	20.85	22.31	25.71	85.27	89.36	89.83	90.70	91.32	91.98
	Joint-GCG(AAAI 26)	22.80	27.03	21.27	18.65	18.49	21.30	93.75	93.81	90.30	94.29	88.64	89.37
	Ours	17.21	18.36	14.19	16.33	16.16	14.07	92.63	94.89	91.05	93.70	92.00	91.55
MS-MARCO	Clean	60.01	54.72	50.23	51.43	47.96	56.29	—	—	—	—	—	—
	PoisonedRAG-BB(USENIX 25)	30.17	25.81	27.62	28.93	30.60	33.95	85.30	88.76	85.54	83.89	90.01	89.72
	The RAG Paradox(EMNLP 25)	27.54	21.60	21.14	23.35	22.08	26.39	89.28	93.25	90.66	90.81	85.43	88.92
	Joint-GCG(AAAI 26)	20.78	22.12	19.85	19.50	24.11	22.28	90.10	87.93	90.26	91.78	92.01	87.65
	Ours	19.90	23.69	20.11	22.58	20.12	20.97	90.07	90.96	92.85	89.74	90.56	90.71

TABLE II: Acc and ASR in RAG Systems with Chunking, Retrieval, and Reranking Pipelines.

Dataset	Method	Accuracy ($\downarrow$ better)						ASR ($\uparrow$ better)					
		GPT-4o	DeepSeek-R1	Qwen3-Max	Qwen2.5-7B	Vicuna-7B	LLaMA-2-7B	GPT-4o	DeepSeek-R1	Qwen3-Max	Qwen2.5-7B	Vicuna-7B	LLaMA-2-7B
NQ	Clean	45.09	46.33	58.15	40.89	41.13	37.50	—	—	—	—	—	—
	PoisonedRAG-BB(USENIX 25)	40.27	43.86	58.07	41.10	39.97	38.16	13.25	12.05	12.33	14.81	13.79	13.64
	The RAG Paradox(EMNLP 25)	42.15	45.07	56.13	40.25	40.09	36.70	15.13	23.98	23.04	14.17	15.06	13.89
	Joint-GCG(AAAI 26)	37.98	44.15	57.06	42.30	39.75	35.03	32.16	27.71	31.82	33.07	33.29	33.18
	Ours	13.59	15.18	15.20	14.16	14.92	15.78	92.19	95.88	94.07	91.53	94.76	93.09
HotpotQA	Clean	50.18	50.75	51.16	52.30	46.92	48.73	—	—	—	—	—	—
	PoisonedRAG-BB(USENIX 25)	49.33	48.52	49.96	52.55	47.04	48.91	16.71	15.02	15.85	14.91	16.16	15.98
	The RAG Paradox(EMNLP 25)	42.15	47.73	46.25	51.10	47.51	49.03	15.25	16.17	17.22	15.18	15.72	17.03
	Joint-GCG(AAAI 26)	39.76	42.19	45.37	52.08	48.75	45.60	34.75	35.99	27.01	36.27	36.85	27.18
	Ours	17.39	19.78	16.17	14.92	18.69	18.93	91.76	95.01	92.75	91.36	90.87	90.12
MS-MARCO	Clean	57.85	55.61	51.36	47.95	49.18	55.64	—	—	—	—	—	—
	PoisonedRAG-BB(USENIX 25)	51.13	57.90	49.87	49.12	47.69	53.26	15.31	15.85	16.77	16.02	15.98	16.15
	The RAG Paradox(EMNLP 25)	47.75	56.91	50.22	46.35	45.78	51.03	15.70	15.18	16.93	15.86	15.17	16.39
	Joint-GCG(AAAI 26)	45.39	47.88	52.07	45.62	43.13	52.97	35.75	35.43	26.57	27.02	35.95	28.08
	Ours	18.51	16.34	17.78	19.21	18.03	16.72	89.15	90.70	91.57	90.23	88.79	90.30

the reranker.

When white-box surrogate reranker access is unavailable, we substitute $\mathcal{L}_{rr}$ with a cross-encoder trained on MS-MARCO [30], which we find sufficient due to strong cross-reranker transferability across model families of the same architecture class (see Table IV).

2) *Adversarial Passage Composition*: To further improve reranker survivability, CRCP organizes poisoning documents as a collection of semi-independent adversarial passages.

Each passage is designed to remain meaningful even if retrieved individually after chunking.

Concretely, poisoning documents are constructed using the following structure:

- 1) query-related context introduction,
- 2) locally self-contained explanation,
- 3) adversarial target guidance,
- 4) semantically coherent supporting content.

This structure improves both retrieval relevance and reranking compatibility.

3) *Multi-query Generalization*: Practical poisoning attacks should generalize across semantically related queries.

To improve transferability, CRCP optimizes poisoning passages jointly over multiple query variants:

$$\mathcal{L}_{multi} = \mathbb{E}_q \sim \mathcal{Q}[\mathcal{L}_{CRCP}(q)]. \quad (22)$$

This encourages adversarial passages to remain effective for paraphrased or semantically similar queries.

V. EXPERIMENT

A. Experimental Setups

Our experimental setup is as follows:

Datasets. We evaluate our method on widely used open-domain question answering benchmarks commonly adopted in Retrieval-Augmented Generation (RAG) research. Specifically, we use Natural Questions (NQ) [31], HotpotQA [32], and MS-MARCO [30], which contain diverse factual and multi-hop queries requiring external knowledge retrieval.

LLMs. To evaluate the robustness of poisoning attacks across different generation backbones, we conduct experiments using multiple widely adopted large language models, including: GPT-4o, DeepSeek-R1, Qwen3-Max, Qwen2.5-7B, Vicuna-7B and LLaMA-2-7B.

Embedding Model. For document indexing and retrieval representation, we employ embedding model BGE-m3 to each retriever configuration.

Chunking Configurations. To evaluate robustness against retrieval granularity changes, we test multiple chunking configurations with different chunk sizes and chunking style. Specifically, chunk sizes are selected from $\{128, 256, 512\}$ characters. The chunking styles are categorized into the following two types [33]:

- **Early Chunking.** Documents are segmented into text chunks, each of which is processed independently by the embedding model. Token-level embeddings generated for each chunk are then aggregated via mean pooling to obtain a single chunk representation.

TABLE III: ASR (% , $\uparrow$ better) of CRCP under different chunking styles and chunk sizes on three datasets. **Bold** denotes the highest value in each row. DS-R1: DeepSeek-R1; Q3-Max: Qwen3-Max; Q2.5: Qwen2.5-7B; Vic: Vicuna-7B; LL2: LLaMA-2-7B.

Style	Size	Retriever	Reranker	NQ						HotpotQA						MS-MARCO					
				GPT-4o	DS-R1	Q3-Max	Q2.5	Vic	LL2	GPT-4o	DS-R1	Q3-Max	Q2.5	Vic	LL2	GPT-4o	DS-R1	Q3-Max	Q2.5	Vic	LL2
Early	128	Contriever	MonoT5	80.17	80.53	83.94	82.26	81.71	80.49	81.07	83.13	82.06	81.22	84.35	83.94	83.13	82.36	82.65	80.16	83.27	81.32
		Contriever	BGE-Base	81.08	80.35	81.89	83.12	80.96	82.77	81.55	80.31	83.77	83.24	80.90	81.33	81.70	82.55	82.46	84.13	81.75	80.23
		ANCE	MonoT5	81.03	83.44	80.58	83.32	82.64	84.81	81.03	83.66	81.85	82.04	84.21	84.87	81.60	84.32	84.61	83.29	81.47	80.31
		ANCE	BGE-Base	83.07	80.39	81.85	83.21	82.49	83.73	83.15	83.34	81.55	81.27	83.95	83.66	82.00	84.08	81.35	82.27	82.19	83.41
		DPR	MonoT5	84.96	80.01	84.98	82.67	80.15	84.60	85.69	81.34	81.78	83.54	83.50	82.09	84.09	82.18	81.71	82.30	82.25	83.69
		DPR	BGE-Base	83.44	80.30	82.58	80.82	84.27	80.73	81.67	81.93	83.25	83.56	81.75	81.38	81.07	81.22	80.67	84.09	83.21	83.83
	256	Contriever	MonoT5	85.31	88.63	87.94	85.19	86.78	86.37	86.71	86.39	85.43	85.22	87.47	85.61	85.07	85.16	85.39	85.92	84.71	82.98
		Contriever	BGE-Base	87.85	86.46	87.02	85.57	86.29	87.64	84.57	81.39	87.60	83.25	84.11	86.17	81.63	80.31	87.28	84.51	83.34	84.27
		ANCE	MonoT5	85.93	87.81	87.18	86.14	86.70	87.50	85.52	87.88	87.13	82.40	85.51	80.19	83.69	81.61	88.26	85.39	83.43	86.60
		ANCE	BGE-Base	87.23	86.71	85.48	85.96	85.15	86.89	85.28	88.62	83.00	85.17	86.52	85.87	83.90	85.21	86.78	88.95	82.35	84.79
		DPR	MonoT5	86.07	87.54	87.23	85.82	87.39	87.76	85.36	85.51	87.66	81.53	84.37	87.19	82.01	80.19	83.20	81.47	82.36	84.69
		DPR	BGE-Base	86.44	85.92	87.11	87.68	86.07	87.55	85.38	81.92	82.10	81.49	85.73	85.34	85.77	86.31	83.10	84.89	81.65	88.70
	512	Contriever	MonoT5	90.81	92.13	91.36	93.52	94.36	93.21	90.11	93.27	91.72	92.35	92.77	93.81	90.77	92.10	90.16	94.21	93.96	93.37
		Contriever	BGE-Base	92.31	92.11	93.49	91.21	93.69	91.33	92.01	92.45	93.69	91.16	92.74	90.30	92.85	92.51	91.29	94.24	93.59	90.38
		ANCE	MonoT5	94.10	90.06	90.17	89.02	92.33	89.07	93.78	91.15	90.14	89.77	90.26	92.57	91.40	91.67	92.15	88.46	91.23	91.05
		ANCE	BGE-Base	92.16	90.37	88.32	88.69	90.11	93.35	92.63	92.59	89.16	90.62	90.57	91.49	91.12	93.57	91.39	88.70	91.08	92.16
		DPR	MonoT5	89.17	90.36	91.29	91.76	92.33	89.46	91.37	91.99	91.06	90.88	89.47	90.19	91.87	89.20	91.24	92.16	92.01	93.68
		DPR	BGE-Base	93.59	88.77	90.36	88.60	89.93	92.62	92.78	91.08	90.60	92.14	89.65	91.58	91.96	90.37	90.29	89.21	88.90	91.81
Late	128	Contriever	MonoT5	70.28	70.54	72.97	71.41	73.90	70.67	72.17	73.45	72.06	71.30	70.66	72.41	70.19	73.50	72.45	71.15	74.93	72.78
		Contriever	BGE-Base	73.24	76.49	72.17	75.71	75.59	74.89	71.24	71.99	72.37	70.67	76.34	72.78	71.85	73.42	72.72	70.46	73.29	72.90
		ANCE	MonoT5	71.49	73.08	73.54	74.95	71.11	71.77	73.35	71.27	72.64	72.55	71.06	75.40	71.20	73.67	71.32	77.40	73.28	70.34
		ANCE	BGE-Base	70.34	73.91	72.49	70.69	71.96	74.73	70.47	75.13	74.07	73.29	76.91	74.35	74.23	73.87	73.21	72.34	72.16	75.17
		DPR	MonoT5	73.07	74.62	70.95	73.25	72.46	73.63	75.27	76.12	72.33	74.21	73.29	73.17	72.50	73.69	72.15	71.06	76.37	73.90
		DPR	BGE-Base	74.19	71.57	71.83	74.19	73.88	70.65	72.12	73.75	72.33	70.47	70.31	73.79	75.88	75.52	73.81	71.26	73.21	72.63
	256	Contriever	MonoT5	80.23	81.67	80.94	83.41	81.78	80.55	83.29	81.78	82.21	82.36	81.65	85.55	80.75	80.27	83.46	82.37	84.05	83.65
		Contriever	BGE-Base	82.09	82.36	83.85	80.17	80.91	81.48	81.46	82.31	81.85	80.93	80.81	82.35	82.90	82.73	83.08	82.79	85.90	83.28
		ANCE	MonoT5	80.72	83.03	81.56	80.99	81.94	83.19	82.70	82.05	81.59	84.16	81.22	82.00	82.04	83.70	85.26	83.91	80.40	81.93
		ANCE	BGE-Base	83.67	85.00	81.52	83.79	84.62	80.49	81.27	83.09	81.26	83.15	82.92	83.91	86.37	80.28	80.72	83.00	84.09	83.16
		DPR	MonoT5	81.96	82.63	81.52	80.60	81.77	83.46	84.15	83.39	85.16	80.71	80.29	83.91	81.27	81.33	81.02	85.80	83.27	81.43
		DPR	BGE-Base	82.16	81.24	82.09	85.71	84.90	81.95	80.05	81.80	85.09	85.11	82.70	83.91	80.57	80.94	85.19	83.21	83.60	82.95
	512	Contriever	MonoT5	88.71	86.90	91.00	90.93	92.67	92.01	90.06	88.56	90.70	91.33	88.15	91.21	88.10	89.03	92.39	92.10	92.47	91.68
		Contriever	BGE-Base	90.36	90.11	93.02	90.19	90.83	88.11	93.76	91.15	92.08	89.17	92.50	89.17	91.36	88.18	90.49	90.12	90.77	90.10
		ANCE	MonoT5	89.16	90.79	90.18	93.77	91.65	91.30	89.15	90.28	90.36	93.52	92.38	90.10	90.63	90.25	92.17	91.07	91.42	92.10
		ANCE	BGE-Base	89.86	93.29	89.17	90.08	91.65	91.47	88.75	90.27	89.88	90.35	92.67	90.41	90.75	89.91	89.75	93.20	90.51	94.21
		DPR	MonoT5	89.73	93.11	93.07	92.25	90.61	90.32	89.52	89.13	90.05	91.20	93.11	93.42	90.33	92.05	94.17	92.60	93.11	93.40
		DPR	BGE-Base	90.14	93.68	88.96	91.21	93.05	92.60	90.25	90.67	88.55	93.41	92.19	89.60	92.90	89.73	92.10	92.07	92.01	93.05

TABLE IV: Ablation study of loss components in CRCP. ASR denotes Attack Success Rate (%).

Variant	$\mathcal{L}_{ret}$	$\mathcal{L}_{rr}$	$\mathcal{L}_{coh}$	$\mathcal{L}_{boundary}$	ASR (GPT-4o)	ASR (LLaMA-2-7B)
Full CRCP	✓	✓	✓	✓	92.19	93.09
w/o $\mathcal{L}_{rr}$	✓	×	✓	✓	55.8	56.1
w/o $\mathcal{L}_{coh}$	✓	✓	×	✓	43.4	44.2
w/o $\mathcal{L}_{boundary}$	✓	✓	✓	×	61.3	62.7
$\mathcal{L}_{ret}$ only	✓	×	×	×	14.2	15.3

- **Late Chunking.** Late chunking postpones document segmentation until after embedding, the entire document is first processed by the embedding model to obtain token-level embeddings. These embeddings are subsequently divided into chunks.

This setting simulates practical deployment variations across real-world RAG systems.

Retrievers. We select three widely-used dense retrievers: Contriever, ANCE and DPR.

Rerankers. To investigate the impact of reranking on poisoning robustness, we evaluate two widely adopted reranker models, MonoT5 and BGE-Reranker-Base.

Evaluation Metrics. We evaluate attack effectiveness using the following metrics:

- **Accuracy (Acc).** The proportion of queries where the correct answer span appears in the system’s generated response. This captures overall performance degradation under attack.

TABLE V: Hyperparameter sensitivity analysis of CRCP on NQ (ASR %). Each block varies one λ while fixing the others at the default values $(\lambda_1, \lambda_2, \lambda_3, \lambda_4) = (1.0, 2.0, 0.5, 0.3)$. **Bold/ shaded** rows denote the default configuration. Results are averaged over 3 random seeds; std. $< 1.2\%$.

Block	Varied Weight	Value	GPT-4o	LLaMA-2-7B	Mean	Remark
λ_1	Retrieval ($\mathcal{L}_{ret}$)	0.25	78.3	79.1	78.7	Under-weights retrieval stage
		0.50	84.7	85.3	85.0	
		1.00	92.2	93.1	92.7	Default
		2.00	91.5	92.4	92.0	Marginal degradation
		4.00	89.8	90.6	90.2	Suppresses $\mathcal{L}_{rr}$
λ_2	Reranker ($\mathcal{L}_{rr}$)	0.50	45.3	46.1	45.7	Reranker severely under-weighted
		1.00	71.2	72.0	71.6	
		2.00	92.2	93.1	92.7	Default
		3.00	91.8	92.6	92.2	Stable plateau
		5.00	89.1	90.0	89.6	Over-emphasis on reranker
λ_3	Coherence ($\mathcal{L}_{coh}$)	0.10	80.1	81.3	80.7	Low coherence impairs reranker
		0.25	87.4	88.0	87.7	
		0.50	92.2	93.1	92.7	Default
		1.00	91.6	92.3	92.0	Robust to moderate increase
		2.00	90.3	91.0	90.7	Crowds out retrieval term
λ_4	Boundary ($\mathcal{L}_{boundary}$)	0.10	84.1	85.2	84.7	Sensitive to chunk boundaries
		0.20	88.9	89.7	89.3	
		0.30	92.2	93.1	92.7	Default
		0.50	91.5	92.1	91.8	Slightly over-regularised
		1.00	88.3	89.1	88.7	Over-penalises score variance

Note: λ_2 exhibits the highest sensitivity among all four weights, consistent with Table IV where removing $\mathcal{L}_{rr}$ causes the largest single-component ASR drop. CRCP maintains ASR $\geq 91.5\%$ when $\lambda_2 \in [1.0, 3.0]$, confirming robust performance in a wide neighbourhood of the default.

- **Attack Success Rate (ASR).** The proportion of target queries for which the generated responses match the attacker-specified target answers.

All reported results are averaged over multiple random

TABLE VI: Performance Comparison of CRCP under Different Defense Strategies

Model	Method	NQ	HotpotQA	MS-MARCO
Qwen2.5-7B	VanillaRAG	91.53	91.36	90.23
	InstructRAG	80.94	77.32	72.10
	ASTUTERAG	78.36	73.72	70.71
	TrustRAG	73.29	70.68	68.43
	SeconRAG	77.17	70.85	69.04
LLaMA-2-7B	VanillaRAG	93.09	90.12	90.30
	InstructRAG	81.26	80.37	80.09
	ASTUTERAG	77.49	76.30	76.29
	TrustRAG	79.13	72.34	77.75
	SeconRAG	78.85	70.58	78.93
GPT-4o	VanillaRAG	92.19	91.76	89.15
	InstructRAG	84.11	77.32	82.10
	ASTUTERAG	73.07	71.49	70.13
	TrustRAG	69.02	68.85	77.09
	SeconRAG	78.41	74.72	77.85
DeepSeek-R1	VanillaRAG	95.88	95.01	90.70
	InstructRAG	86.26	79.01	83.68
	ASTUTERAG	77.30	71.25	74.97
	TrustRAG	75.82	69.07	72.41
	SeconRAG	73.76	65.34	67.82

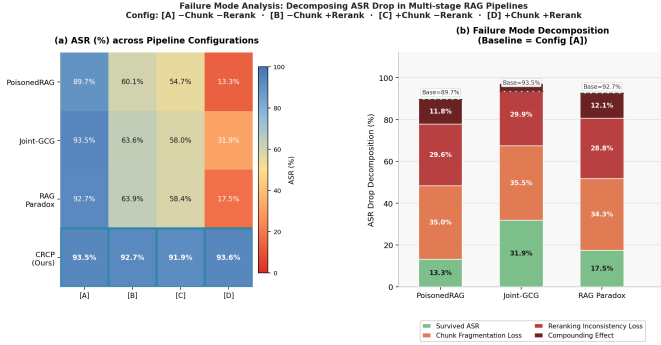


Fig. 4: Failure mode analysis of corpus poisoning attacks

seeds.

B. Main Results

We first evaluate the attack performance under a simplified setting commonly adopted in prior studies, where the RAG system consists only of a retriever and a generator, without chunking or reranking modules. The results, reported in Table I, show that our proposed CRCP achieves attack performance comparable to several state-of-the-art poisoning methods, demonstrating the effectiveness of our approach.

We then extend the evaluation to a more realistic RAG setting that incorporates the full pipeline of chunking, retrieval, reranking, and generation. Under this setting, the experimental results differ substantially. As shown in Table II, CRCP consistently maintains strong attack performance, achieving attack success rates above 88% across all three datasets. In contrast, the effectiveness of existing methods degrades significantly.

Table III compares the attack performance of CRCP under different chunking styles and chunk size configurations. The results demonstrate that CRCP consistently maintains strong

attack effectiveness across diverse chunking configurations, indicating its robustness to variations in chunking strategies.

The observed ASR gap between Early and Late Chunking (approximately 10 percentage points at chunk size 128) is a mechanistically expected consequence of their distinct embedding paradigms. Early Chunking encodes each chunk independently, preserving locally optimized adversarial signals with high fidelity, whereas Late Chunking conditions chunk embeddings on the full document context, inherently diluting locally concentrated adversarial semantics. Despite this suppression effect, CRCP maintains ASR above 70% under Late Chunking across all configurations—substantially exceeding the 13–35% ASR of existing baselines even under the comparatively less challenging retrieval-only pipeline confirming robust adversarial effectiveness across both chunking paradigms.

These results indicate that CRCP is effective in realistic RAG pipelines involving chunking, retrieval, and reranking, thereby posing a substantially greater practical threat.

C. Ablation Studies

To further investigate the effectiveness of CRCP, we conduct a series of ablation studies. Component-level ablation (Table IV) removes each loss term from Eq.19 individually while keeping the full pipeline intact. Removing $\mathcal{L}_{rr}$ causes the most dramatic ASR drop (from 92.19% to 55.8% on NQ/GPT-4o), confirming that reranker consistency is the single most critical factor for attack survival in multi-stage pipelines. Removing $\mathcal{L}_{coh}$ reduces ASR by 19 points, while removing $\mathcal{L}_{boundary}$ incurs an 11-point drop, showing that local coherence and boundary robustness each contribute meaningfully but are secondary to reranker alignment. The retrieval-only variant ($\mathcal{L}_{ret}$ only) collapses to 14.2% ASR, matching prior methods and validating that retrieval-stage optimization alone is insufficient.

Table V reports the sensitivity of CRCP to the four loss weights. Overall, CRCP remains stable across a wide range of hyperparameter values, indicating good robustness to parameter tuning. Among all components, the reranker weight λ_2 has the greatest impact on attack effectiveness. Specifically, decreasing λ_2 from 2.0 to 0.5 reduces ASR from 92.7% to 45.7%, confirming that rerank consistency is the key factor driving successful poisoning. In contrast, varying λ_1 , λ_3 , and λ_4 results in only moderate performance changes, suggesting that retrieval alignment, coherence preservation, and boundary regularization mainly serve as complementary objectives. These results validate the effectiveness of the default configuration and demonstrate that CRCP achieves consistently high ASR without requiring extensive hyperparameter tuning.

In Figure 4, CRCP exhibits negligible degradation across all four configurations. These results directly validate the retrieval granularity mismatch hypothesis and motivate CRCP’s multi-objective design.

D. Defense Experiments

We evaluate the effectiveness of CRCP against several representative and state-of-the-art RAG defense strategies.

As shown in Table VI, CRCP effectively bypasses existing RAG defenses due to its design choices. InstructRAG, which relies on the LLM’s intrinsic knowledge to verify retrieved content, is partially effective, with ASR only decreasing by 10%, because CRCP passages are highly fluent and factually coherent, leading the LLM to trust them. ASTUTERAG and TrustRAG, which use cross-document consistency voting, fail since CRCP only requires a single high-quality passage to rank first in reranking, eliminating the need for multiple documents to collude. SeconRAG, which detects anomalies via semantic clustering, is evaded as CRCP’s locally self-contained passages produce embeddings that overlap with normal passages, rendering cluster-based detection ineffective.

These findings highlight the limitations of existing defenses and demonstrate the need for more robust defense mechanisms against corpus poisoning attacks in modern RAG systems.

VI. CONCLUSION

We revisit corpus poisoning under realistic multi-stage RAG pipelines and identify retrieval granularity mismatch as a fundamental limitation of existing attacks, which degrade severely after chunking and reranking. To address this, we propose CRCP, jointly optimizing retrieval relevance, reranker consistency, and chunk-boundary robustness. Experiments across diverse datasets, retrievers, rerankers, and LLMs demonstrate that CRCP achieves substantially higher attack success rates than existing methods under realistic pipeline settings. Our findings reframe corpus poisoning as a multi-stage retrieval consistency problem and highlight the need for more realistic RAG security evaluation.

REFERENCES

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K ttler, Mike Lewis, Wen-tau Yih, Tim Rockt schel, et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *NeurIPS*, 2020.
- [2] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih, “Dense passage retrieval for open-domain question answering,” in *EMNLP*, 2020.
- [3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al., “Improving language models by retrieving from trillions of tokens,” in *ICML*, 2022.
- [4] Sina Semnani, Violet Yao, Heidi Zhang, and Monica Lam, “Wikichat: Stopping the hallucination of large language model chatbots by few-shot grounding on wikipedia,” in *Findings of EMNLP*, 2023.
- [5] Noor Nashid, Mifta Sintaha, and Ali Mesbah, “Retrieval-based prompt selection for code-related few-shot learning,” in *ICSE*, 2023.
- [6] Shuyan Zhou, Uri Alon, Frank F. Xu, Zhengbao Jiang, and Graham Neubig, “Docprompting: Generating code by retrieving the docs,” in *ICLR*, 2023.
- [7] Harsh Chaudhari, Giorgio Severi, John Abascal, Anshuman Suri, Matthew Jagielski, Christopher A. Choquette-Choo, Milad Nasr, Cristina Nita-Rotaru, and Alina Oprea, “Phantom: General backdoor attacks on retrieval augmented language generation,” *ACM Trans. AI Secur. Priv.*, 2026.
- [8] Sukmin Cho, Soyeong Jeong, Jeongyeon Seo, Taeho Hwang, and Jong C Park, “Typos that broke the rag’s back: Genetic attack on rag pipeline by simulating documents in the wild via low-level perturbations,” in *Findings of EMNLP*, 2024.
- [9] Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou, “Hotflip: White-box adversarial examples for text classification,” in *ACL*, 2018.
- [10] Runqi Sui, Xuejing Yuan, Di Tang, and Baojiang cui, “Ctrlrag: Black-box document poisoning attacks for retrieval-augmented generation of large language models,” in *AAMAS*, 2026.
- [11] Zexuan Zhong, Ziqing Huang, Alexander Wettig, and Danqi Chen, “Poisoning retrieval corpora by injecting adversarial passages,” in *EMNLP*, 2023.
- [12] Muhammad Arslan, Hussam Ghanem, Saba Munawar, and Christophe Cruz, “A survey on rag with llms,” *Procedia computer science*, 2024.
- [13] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li, “A survey on rag meeting llms: Towards retrieval-augmented large language models,” in *SIGKDD*, 2024.
- [14] Michael Glass, Gaetano Rossiello, Md Faisal Mahbub Chowdhury, Ankita Naik, Pengshan Cai, and Alfio Gliozzo, “Re2g: Retrieve, rerank, generate,” in *NAACL*, 2022.
- [15] Jiashuo Sun, Xianrui Zhong, Sizhe Zhou, and Jiawei Han, “Dynamicrag: Leveraging outputs of large language model as feedback for dynamic reranking in retrieval-augmented generation,” in *NeurIPS*, 2025.
- [16] Yihui Yang, Jianhui Ma, Xu An, Zheng Zhang, Mingfan Pan, Wuhong Wang, and Huiming Ding, “Prorank: Progressive context refinement for reliable retrieval-augmented generation,” in *ICASSP*, 2026.
- [17] Tolga  akar and Hakan Emekci, “Maximizing rag efficiency: A comparative analysis of rag methods,” *Natural Language Processing*, 2025.
- [18] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li, “Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases,” *NeurIPS*, 2024.
- [19] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia, “Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models,” in *USENIX Security*, 2025.
- [20] Zhen Tan, Chengshuai Zhao, Raha Moraffah, Yifan Li, Song Wang, Jundong Li, Tianlong Chen, and Huan Liu, “Glue pizza and eat rocks: exploiting vulnerabilities in retrieval-augmented generative models,” in *EMNLP*, 2024.
- [21] Haowei Wang, Rupeng Zhang, Junjie Wang, Mingyang Li, Yuekai Huang, Dandan Wang, and Qing Wang, “Joint-gcg: Unified gradient-based poisoning attacks on retrieval-augmented generation systems,” in *AAAI*, 2026.
- [22] Chanwoo Choi, Jinsoo Kim, Sukmin Cho, Soyeong Jeong, and Buru Chang, “The RAG paradox: A black-box attack exploiting unintentional vulnerabilities in retrieval-augmented generation systems,” in *Findings of EMNLP*, 2025.
- [23] Chunyang Li, Junwei Zhang, Anda Cheng, Zhuo Ma, Xinghua Li, and Jianfeng Ma, “Cpa-rag: Covert poisoning attacks on retrieval-augmented generation in large language models,” *arXiv preprint arXiv:2505.19864*, 2025.
- [24] Xue Tan, Hao Luan, Mingyu Luo, Xiaoyan Sun, Ping Chen, and Jun Dai, “RevPRAG: Revealing poisoning attacks in retrieval-augmented generation through LLM activation analysis,” in *Findings of EMNLP*, 2025.
- [25] Chong Xiang, Tong Wu, Zexuan Zhong, David Wagner, Danqi Chen, and Prateek Mittal, “Certifiably robust RAG against retrieval corruption,” in *ICML 2024 Next Generation of AI Safety Workshop*, 2024.
- [26] Fei Wang, Xingchen Wan, Ruoxi Sun, Jiefeng Chen, and Sercan O Arik, “Astute rag: Overcoming imperfect retrieval augmentation and knowledge conflicts for large language models,” in *ACL*, 2025.
- [27] Zhepei Wei, Wei-Lin Chen, and Yu Meng, “Instructrag: Instructing retrieval-augmented generation via self-synthesized rationales,” in *ICLR*, 2025.
- [28] Huichi Zhou, Kin-Hei Lee, Zhonghao Zhan, Yue Chen, Zhenhao Li, Zhaoyang Wang, Hamed Haddadi, and Emine Yilmaz, “Trustrag: enhancing robustness and trustworthiness in retrieval-augmented generation,” in *AAAI Workshop*, 2026.
- [29] Xiaonan Si, Meilin Zhu, Simeng Qin, Lijia Yu, Lijun Zhang, Shuaitong Liu, Xinfeng Li, Ranjie Duan, Yang Liu, and Xiaojun Jia, “Secon-rag: A two-stage semantic filtering and conflict-free framework for trustworthy rag,” *NeurIPS*, 2025.
- [30] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al., “Ms marco: A human generated machine reading comprehension dataset,” *arXiv preprint arXiv:1611.09268*, 2016.
- [31] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al., “Natural questions: a benchmark

for question answering research,” *Transactions of the Association for Computational Linguistics*, 2019.

- [32] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning, “Hotpotqa: A dataset for diverse, explainable multi-hop question answering,” in *EMNLP*, 2018.
- [33] Carlo Merola and Jaspinder Singh, “Reconstructing context: Evaluating advanced chunking strategies for retrieval-augmented generation,” in *International Workshop on Knowledge-Enhanced Information Retrieval*. Springer, 2025.